

Solution Architecture

Date	2 October 2026	Team ID	SWTID-2026-5625
Project Name	FitBuddy	Maximum Marks	

Solution Architecture Diagram

High-level architecture of **FitBuddy**, an AI-powered fitness assistant. The user interacts with Jinja2-rendered pages served by a FastAPI backend, which sends requests to the Google Gemini API and returns personalised responses. The application is deployed locally.

1. Presentation / Client Layer

Web Browser UI (HTML, CSS, JavaScript) rendered with Jinja2 templates



2. Application Server / Routing Layer

FastAPI (Python), Uvicorn server, route handling & user-input validation



3. Core Logic Service & Template Service

Fitness prompt building, response processing, FitBuddy main logic (Python)
Jinja2 dynamic templates, form handling (FastAPI)
External API Integration: Google Gemini API (Generative AI model)



4. Data / Configuration Layer

Environment file (.env) for Gemini API key, static files (CSS/JS), in-memory session data — no external database

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Web interface where users enter fitness details and goals and view AI-generated responses. Designed to be simple and user-friendly.	HTML, CSS, JavaScript, Jinja2 templates

Application Server / Routing (API Gateway role)	Receives browser requests, routes them to the correct handler, validates user input, and returns rendered pages. Template service connects frontend templates with the backend.	FastAPI, Uvicorn, Python
Core Logic & AI Services (Microservices role)	Core logic builds prompts from user input and calls the Gemini API to generate fitness guidance, then processes the response.	Python, Jinja2, Google Gemini API (google-generativeai SDK)
Data / Configuration Layer (Database role)	Stores configuration such as the Gemini API key and serves static assets. The app is deployed locally and does not use a persistent database.	Environment variables (.env), static files, local deployment